

Heaven's Light is Our Guide
Rajshahi University of Engineering and Technology



Course Code
ECE 4124

Course Title
Digital Signal Processing Sessional

Experiment Date: July 5, 2025,
Submission Date: July 21, 2025

Lab Report 1:
Study of Impulse, Step, and Ramp signals and their inter-relationships

Submitted to
Md. Faysal Ahamed
Lecturer
Dept of ECE, Ruet

Submitted by
Md. Tajim An Noor
Roll: 2010025

Contents

Theory	1
Used Tools	2
Code & Output	3
Unit Impulse Signal	3
Unit Step Signal	3
Unit Ramp Signal	4
Conversion	5
Discussion	7
Conclusion	7
References	7

Study of Impulse, Step, and Ramp signals and their inter-relationships

Theory

In digital signal processing, several basic discrete-time signals serve as the building blocks for analyzing and synthesizing more complex signals [1]. The three most important among these are:

1. Unit Impulse Signal ($\delta[n]$):

The unit impulse signal, $\delta[n]$, is defined as:

$$\delta[n] = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases}$$

- Serves as the identity element in convolution operations [1].
- Has a value of 1 only at $n = 0$ and 0 elsewhere.
- Commonly used to determine a system's impulse response.

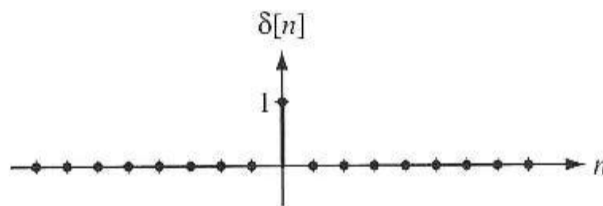


Figure 1: Unit Impulse Signal $\delta[n]$

2. Unit Step Signal ($u[n]$):

The unit step signal, $u[n]$, is defined as:

$$u[n] = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0 \end{cases}$$

- Switches from 0 to 1 at $n = 0$ and remains at 1 for all subsequent n .
- Can be interpreted as the running sum of the impulse signal: $u[n] = \sum_{k=-\infty}^n \delta[k]$ [1].



Figure 2: Unit Step Signal $u[n]$

3. Unit Ramp Signal ($r[n]$):

The unit ramp signal, $r[n]$, is defined as:

$$r[n] = \begin{cases} 0, & n < 0 \\ n, & n \geq 0 \end{cases}$$

- Increases linearly with n starting from zero.
- Represents the cumulative sum (discrete integration) of the unit step signal: $r[n] = \sum_{k=-\infty}^n u[k]$ [1].

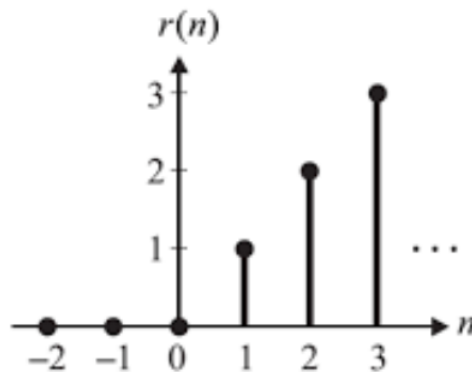


Figure 3: Ramp Signal $u[n]$

Used Tools

- Matlab
- LaTeX

Code & Output

Unit Impulse Signal

```
1 % Length of impulse response
2 N = 11; % Total number of samples
3 % Generate unit impulse using impz
4 x = impz(1, 1, N); % delta[n] from n = 0 to N-1
5 % Time index
6 n = 0:N-1;
7 % Plot the impulse
8 stem(n, x, 'filled', 'LineWidth', 2);
9 grid on;
10 xlabel('n (discrete time)');
11 ylabel('Amplitude');
12 title('Discrete Unit Impulse Signal (delta[n])');
13 ylim([-0.2 1.2]);
```

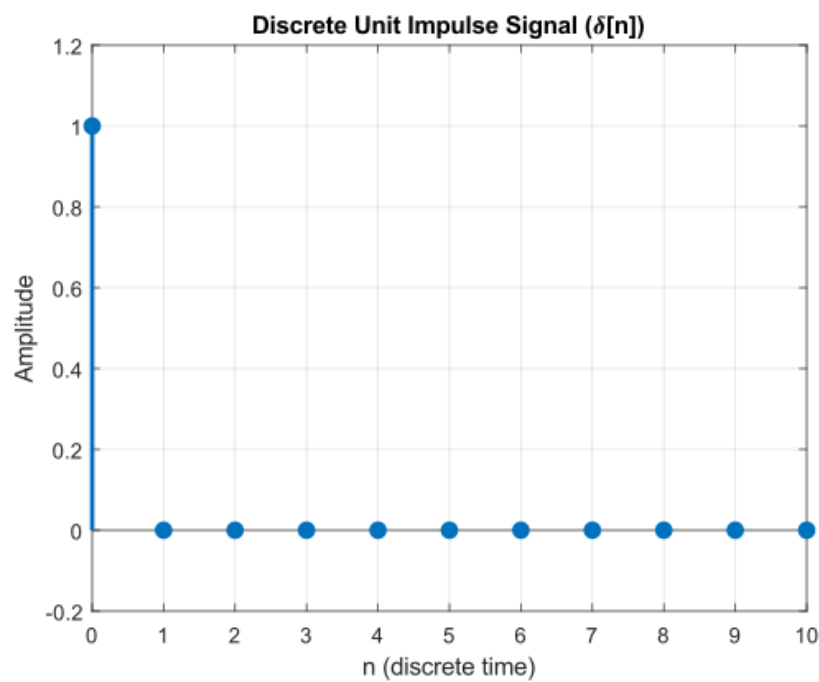


Figure 4: Unit Impulse Signal $u[n]$

Unit Step Signal

```
1 % Define discrete time index
2 n = -5:5; % From -5 to 5
3 % Generate discrete unit step signal
4 x = double(n >= 0); % u[n] = 1 for n >= 0, 0 otherwise
5 % Plot the step signal
```

```

6  stem(n, x, 'filled', 'LineWidth', 2);
7  grid on;
8  xlabel('n (discrete time)');
9  ylabel('Amplitude');
10 title('Discrete Unit Step Signal u[n]');
11 ylim([-0.2 1.2]);

```

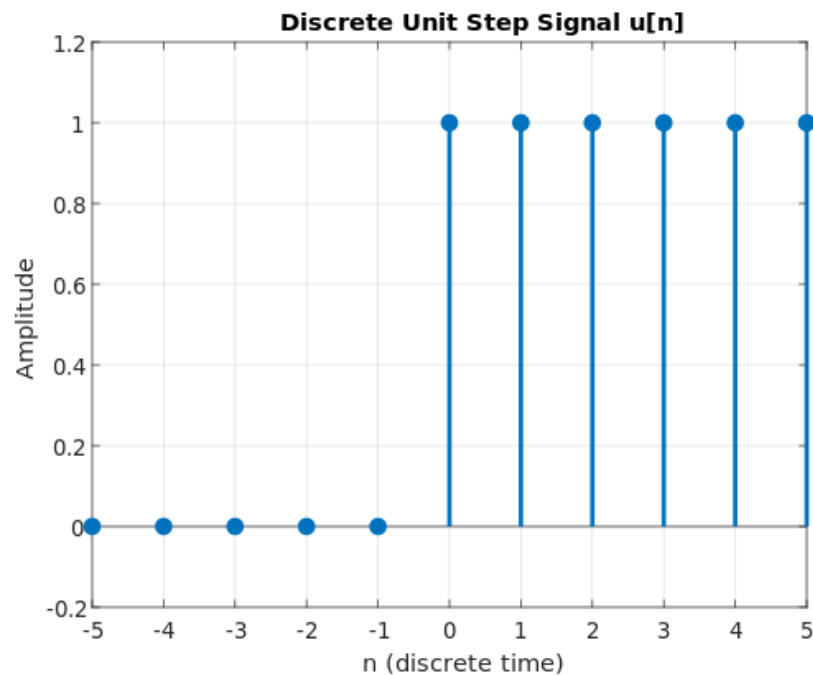


Figure 5: Unit Step Signal $u[n]$

Unit Ramp Signal

```

1  % Define discrete time index
2  n = -5:10; % From -5 to 10 (includes negative range)
3  % Generate discrete unit ramp signal
4  x = max(0, n); % x[n] = n for n >= 0, 0 otherwise
5  % Plot the ramp signal
6  stem(n, x, 'filled', 'LineWidth', 2);
7  grid on;
8  xlabel('n (discrete time)');
9  ylabel('Amplitude');
10 title('Discrete Unit Ramp Signal r[n]');

```

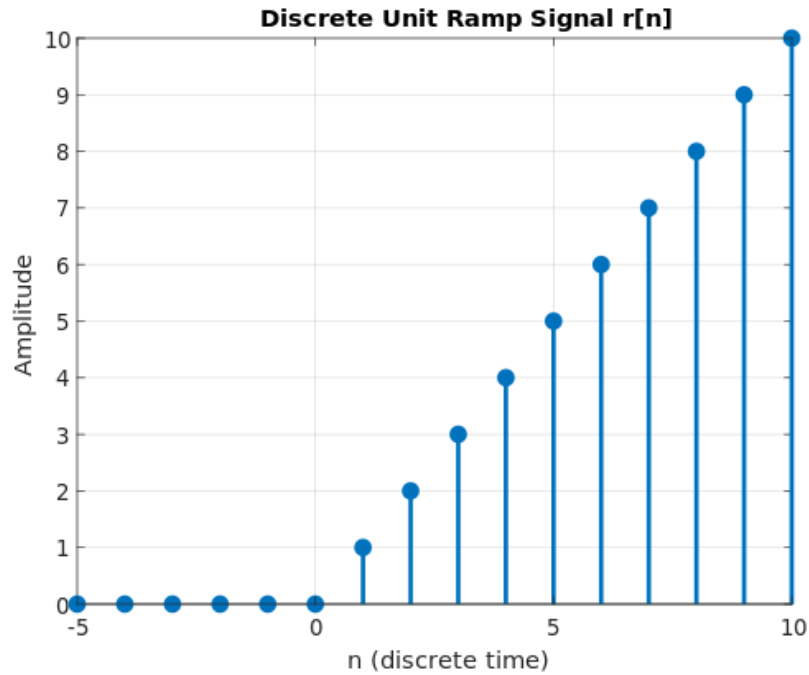


Figure 6: Ramp Signal $r[n]$

Conversion

```

1  % Define discrete time
2  N = 21;
3  n = 0:N-1;
4  % Generate unit step signal u[n]
5  step_signal = double(n >= 0);
6  % Convert step to ramp using cumulative sum
7  ramp_signal = cumsum(step_signal);
8  % Convert ramp back to step using difference
9  recovered_step = [ramp_signal(1), diff(ramp_signal)];
10 % Step to Impulse: difference of step signal
11 impulse_from_step = [step_signal(1), diff(step_signal)];
12 % Impulse to Step: cumulative sum of impulse signal
13 step_from_impulse = cumsum(impulse_from_step);
14 % Plotting
15 figure;
16 subplot(5,1,1);
17 stem(n, step_signal, 'filled', 'LineWidth', 2);
18 grid on;
19 title('Original Unit Step Signal u[n]');
20 xlabel('n');
21 ylabel('Amplitude');
22 ylim([-0.2 1.2]);
23 subplot(5,1,2);

```

```

24 stem(n, ramp_signal, 'filled', 'LineWidth', 2);
25 grid on;
26 title('Converted Unit Ramp Signal r[n]');
27 xlabel('n');
28 ylabel('Amplitude');
29 subplot(5,1,3);
30 stem(n, recovered_step, 'filled', 'LineWidth', 2);
31 grid on;
32 title('Recovered Step Signal from Ramp');
33 xlabel('n');
34 ylabel('Amplitude');
35 ylim([-0.2 1.2]);
36 subplot(5,1,4);
37 stem(n, impulse_from_step, 'filled', 'LineWidth', 2);
38 grid on;
39 title('Impulse Signal from Step \delta[n]');
40 xlabel('n');
41 ylabel('Amplitude');
42 ylim([-0.2 1.2]);
43 subplot(5,1,5);
44 stem(n, step_from_impulse, 'filled', 'LineWidth', 2);
45 grid on;
46 title('Recovered Step Signal from Impulse');
47 xlabel('n');
48 ylabel('Amplitude');
49 ylim([-0.2 1.2]);

```

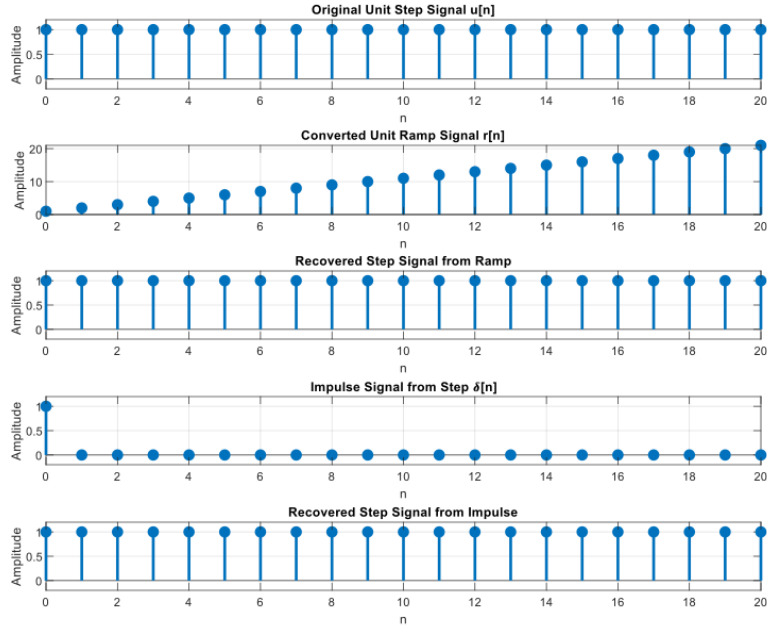



Figure 7: Conversion of Signals

Discussion

In this experiment, we generated and transformed basic discrete-time signals—specifically the impulse, step, and ramp signals—using MATLAB. The “cumsum” function [2] was essential for computing the cumulative sum, enabling us to convert a step signal into a ramp signal and an impulse signal into a step signal, thereby emulating discrete-time integration. Conversely, the “diff” function [3] was used to perform discrete differentiation, allowing us to derive the step signal from the ramp and the impulse signal from the step. The resulting plots from these operations closely aligned with theoretical expectations, validating the correct implementation of these signal relationships. These hands-on conversions provided valuable insight into the mathematical links between the signals and reinforced our understanding of their interdependence.

Conclusion

The experiment effectively illustrated the generation and transformation of fundamental discrete-time signals using essential MATLAB functions. Through examining the characteristics and interconnections of the impulse, step, and ramp signals, we obtained practical understanding of their significance in digital signal processing. Employing functions such as `cumsum` and `diff` enabled us to model discrete-time integration and differentiation, respectively, and confirm theoretical principles via graphical results.

References

- [1] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.
- [2] The MathWorks, Inc., *cumsum: Cumulative sum of array elements*, MATLAB Documentation, 2024, <https://www.mathworks.com/help/matlab/ref/cumsum.html>.
- [3] —, *diff: Differences and Approximate Derivatives*, MATLAB Documentation, 2024, <https://www.mathworks.com/help/matlab/ref/diff.html>.